PERSONAL PORTFOLIO TRACKER

A Command-Line Java Application to Track Stocks, Cryptocurrencies & Mutual Funds

Submitted by	Ahmed Saqlain
Reg. Number	24BAI10947
Course	CSE2006 - Programming in Java
Programme	B.Tech - Computer Science & Engineering (AI&ML)
Academic Year	2025-2026
Institution	VIT UNIVERSITY BHOPAL
Submitted to	Prof. Chour Singh Rajpoot

GitHub: https://github.com/Sakku0saq/Portfolio_tracker

1. Abstract

Managing personal investments across stocks, cryptocurrencies, and mutual funds is genuinely messy. Most students keep a spreadsheet that falls out of date or rely on third-party apps that reveal nothing about the engineering underneath. This project, the Personal Portfolio Tracker, builds the whole thing from scratch in Java: a fully functional command-line application where a user can add holdings, watch profit-and-loss update in real time after entering a current price, export reports in CSV and TXT formats, and have all data persist across sessions through a local SQLite database.

Every major feature maps directly to a CSE2006 syllabus concept. Inheritance drives the asset model, JDBC handles all database interaction, the Collections framework powers search and sort, and a custom exception hierarchy keeps error paths clean. The result is software that actually works and explains itself through its structure.

2. Problem Statement

Most retail investors in India hold assets across at least two or three categories. Someone saving for five years might hold TCS shares, a Bitcoin position, and a couple of SIP-linked mutual funds. Keeping track of total invested capital, current valuation, and unrealised gains requires jumping between apps, bank statements, and memory. There is no single transparent place where all of it lives.

This project addresses that gap: build a tool that tracks all three asset types in one place, persists data without a server or an internet connection, and makes profit-and-loss visible at a glance. The secondary constraint - that it be built in Java as a demonstration of core language concepts - rules out frameworks and forces the design to stay close to the language itself.

Why it matters

- Financial literacy starts with visibility. A student who cannot see their P&L has no feedback loop.
- Existing apps are black boxes. Building a tracker from scratch demystifies how financial software works.
- The project enforces good engineering habits: separation of concerns, exception handling, SQL safety via prepared statements.
- It is fully self-contained - no internet dependency, no paid API, no registration required.

3. Objectives

- Implement a working portfolio management tool supporting Stocks, Cryptocurrencies, and Mutual Funds.
- Demonstrate all major Java concepts from the CSE2006 syllabus within a single coherent application.
- Persist all data across application restarts using JDBC and a local SQLite database.
- Export portfolio reports in both CSV and human-readable TXT formats using Java I/O streams.
- Validate all user inputs and handle error conditions through a proper exception hierarchy.
- Keep the codebase modular - model, DAO, service, and utility layers clearly separated.

4. Approach and Design

4.1 Architecture Overview

The application follows a layered architecture. Main.java renders the CLI menu and delegates all user actions to the service layer. The service layer contains business logic and coordinates between the DAO and the model. The DAO (Data Access Object) owns all JDBC interactions. Model classes are plain data objects. Utility classes handle cross-cutting concerns like database connection management, input validation, and file export.

Layer / Package	Class(es)	Responsibility
Entry Point	Main.java	CLI menu, user interaction loop
Model	Asset, Stock, Crypto, MutualFund, PortfolioSummary	Data representation, OOP hierarchy
DAO	AssetDAO	All SQL - INSERT, SELECT, UPDATE, DELETE
Service	PortfolioService	Business logic, Collections operations
Utility	DatabaseManager, InputValidator, ReportWriter	DB singleton, validation, file I/O
Exception	AssetNotFoundException, InvalidInputException	Typed error propagation

4.2 Object-Oriented Design

The asset model is built around an abstract class Asset. Stock, Crypto, and MutualFund all extend it. This keeps the DAO and service layer free of type-checking conditionals - they work with any Asset reference polymorphically. The abstract method calculateCurrentValue(double price) is the primary polymorphic hook: each subclass computes value differently.

Abstract base + concrete subclasses:

```
public abstract class Asset {
    protected String symbol, name;
    protected double quantity, buyPrice;

    public abstract String getAssetType();
    public abstract double calculateCurrentValue(double price);

    public double getProfitLoss(double currentPrice) {
        return calculateCurrentValue(currentPrice) - (quantity * buyPrice);
    }
}

public class Stock extends Asset {
    private String exchange, sector;
    @Override public String getAssetType() { return "STOCK"; }
    @Override public double calculateCurrentValue(double price) {
        return quantity * price;
```

```
    }
}
```

4.3 Database Design

SQLite was chosen because it requires no server process and the database file is created automatically on first run. The schema uses two tables: assets for all holdings, and price_history to log every price update. This makes it possible to review how valuations changed over time even though the current version does not yet display a full history chart.

Column	Type	Notes
id	INTEGER	Primary key, autoincrement
symbol	TEXT	Ticker / coin / fund code
name	TEXT	Full display name
asset_type	TEXT	STOCK | CRYPTO | MUTUAL_FUND
quantity	REAL	Units held
buy_price	REAL	Purchase price per unit
buy_date	TEXT	ISO date string (YYYY-MM-DD)
currency	TEXT	Default: INR
extra1-3	TEXT	Exchange, blockchain, fund house, expense ratio, etc.

4.4 Singleton Pattern - DatabaseManager

Opening a database connection is expensive, and multiple concurrent connections to the same SQLite file cause locking errors. DatabaseManager implements the Singleton pattern: only one Connection object is ever created per application run, and every class that needs database access receives that same instance.

4.5 Collections Framework

The service layer uses ArrayList to hold lists of assets retrieved from the database, HashMap to cache current prices keyed by asset ID, and Comparator lambdas for in-memory sorting. Sorting by profit-and-loss descending is a single line once the data is in memory:

```
assets.sort((a, b) -> Double.compare(
    b.getProfitLoss(priceMap.get(b.getId())),
    a.getProfitLoss(priceMap.get(a.getId()))
));
```

4.6 File Export - I/O Streams

ReportWriter exports in two formats. The CSV report uses FileWriter wrapped in a BufferedWriter - a character-oriented stream suitable for text data that will be opened in Excel. The TXT report uses FileOutputStream wrapped in OutputStreamWriter with explicit UTF-8 encoding, which demonstrates the byte-to-character bridge and ensures the rupee symbol renders correctly on all platforms.

5. Key Decisions

1. SQLite over plain file storage

Early in development the data persistence layer was a flat CSV file. It worked for simple read/write but made search, filter, and sort expensive to implement correctly. Switching to SQLite added the full power of SQL while keeping the zero-server advantage. The only tradeoff is the JAR dependency - handled by the run scripts automatically.

2. Abstract class over interface for Asset

An interface would have forced each subclass to re-implement common getters and the getProfitLoss() method independently. An abstract class allows shared fields and concrete methods to live in one place. The polymorphic surface is kept small: only getAssetType() and calculateCurrentValue() are abstract.

3. PreparedStatement for all SQL

Every SQL statement in AssetDAO uses PreparedStatement with parameter placeholders. This is not just good production practice - it is the correct way to demonstrate JDBC. SQL injection via string concatenation is a common beginner mistake; the DAO makes clear there is no reason to ever do it.

4. No build tool (Maven / Gradle)

The run scripts download the three required JARs and compile with javac directly. This keeps the project accessible to anyone with a JDK installed, without requiring knowledge of a build system. It also makes the compilation command visible, which helps understanding.

5. CLI over GUI

A Swing GUI would have consumed time that belonged to the core Java concepts the course targets. The CLI keeps focus on data structures, database interaction, and OOP design. The formatted menu output - box-drawing characters, column-aligned tables - is done entirely with Java string formatting, which is itself a useful exercise.

---

6. Features Implemented

Feature	Description
Add Asset	Add stocks, crypto, or mutual fund holdings with full metadata
View Portfolio	Formatted table showing all assets with current P&L
Portfolio Summary	Aggregated totals: invested capital, current value, overall return %
Update Current Price	Manually set market price; triggers P&L recalculation and logs to history
Delete Asset	Remove a holding with a confirmation prompt to prevent accidents
Search	Find assets by symbol or name substring - case-insensitive
Sort by P&L	Ranks all holdings from best performer to worst
Group by Asset Type	Separate views for Stocks, Crypto, and Mutual Funds
Export CSV Report	Machine-readable export for spreadsheet analysis

Export TXT Report	Human-readable formatted report with totals
Persistent Storage	SQLite database auto-created on first run; data survives restarts

7. Challenges Faced

1. Getting JDBC and SQLite to work together

The sqlite-jdbc driver needs to be on the classpath at both compile time and runtime. Early runs threw ClassNotFoundException because the JAR was present during compilation but missing from the runtime classpath command. The fix was to write run scripts that build the classpath string correctly for both operating systems - ':' on Unix and ';' on Windows - and download the JAR automatically if absent.

2. Designing the asset hierarchy without over-engineering

The first draft had a separate DAO class for each asset type. That became repetitive quickly because all three share the same underlying table structure. The solution was a single AssetDAO that uses the asset_type column to distinguish records, plus factory-style logic in the ResultSet mapping to construct the right subclass at read time.

3. Handling type-specific metadata cleanly

Each asset type has specific fields: a stock has an exchange and a sector, a crypto has a blockchain and a wallet address, a mutual fund has a fund house and an expense ratio. Rather than adding separate tables per type (which would complicate every query), the schema uses three generic extra columns. Subclasses interpret these fields in their own constructors.

4. Cross-platform encoding in file export

Writing the rupee symbol to a file exposed a difference between how Windows and Linux handle default stream encoding. The fix - wrapping FileOutputStream in an OutputStreamWriter with StandardCharsets.UTF_8 explicitly specified - resolved garbled output on Windows and made the code a useful illustration of why byte and character streams are separate concepts.

5. Input validation without a GUI

CLI applications cannot block invalid input the way a form field does. Every numeric entry - price, quantity, asset ID - needed a try-catch around Integer.parseInt() or Double.parseDouble(), with a retry loop that does not crash the application. InputValidator was extracted into its own utility class to keep this boilerplate out of Main.java.

8. What I Learned

This project covered a lot of ground and the learning was less about individual language features and more about how they fit together.

- OOP is most useful when it removes duplication. The abstract Asset class becomes interesting when the DAO handles all three asset types without a single if-type branch.
- SQL and Java talk to each other in a specific way and it has to be done right. PreparedStatement is not optional - it is the correct way to pass data to a database from Java.
- Error handling is part of the design, not an afterthought. Custom exceptions make the calling code readable and error messages meaningful.
- Character encoding is a real problem. The rupee symbol breaking on Windows was the first time an encoding issue caused a visible bug - and fixing it required understanding the stream hierarchy

properly.

- Build automation matters even in small projects. The run scripts saved hours of explaining setup and made the project easy to demo on any machine.
- Separation of concerns pays off during debugging. When the sort feature returned wrong results, the bug was in PortfolioService. It took two minutes to find because the layers were cleanly separated.

Beyond Java specifics, this project was an exercise in finishing something. The initial design had more planned features - a price fetch from a public API, a histogram of portfolio value over time, allocation pie charts. These were deliberately cut because they would have shifted focus away from the core requirements. Deciding what not to build, and delivering what remained cleanly, was its own kind of learning.

9. Testing

Testing was done manually against a fixed set of scenarios. The README provides a sample dataset (TCS, INFY, BTC, ETH, HDFC-TOP100) that covers all three asset types and exercises every menu option in sequence.

Test Scenario	Expected Outcome	Result
Add stock with valid inputs	Asset saved, visible in portfolio view	Pass
Add asset with empty symbol	InvalidInputException, user re-prompted	Pass
Update price to 0 or negative	Validation error, price unchanged	Pass
Delete non-existent asset ID	AssetNotFoundException shown	Pass
Search by partial symbol	Matching assets listed	Pass
Export CSV then open in Excel	Columns align, rupee symbol correct	Pass
Restart app and re-check data	All assets persist in SQLite	Pass
Sort by P&L with mixed gains/losses	Correct descending order	Pass

10. Future Improvements

- Live price fetch - integrate a free public API (Yahoo Finance, CoinGecko) to pull current prices automatically.
- Price history chart - display a simple ASCII line graph of value-over-time from the price_history table.
- Allocation breakdown - show portfolio split by asset type as percentages.
- Multi-user support - add a simple login layer so different users can maintain separate portfolios on the same machine.
- GUI front-end - a JavaFX interface would make the data significantly more scannable.
- Unit tests - replace manual test scenarios with JUnit test cases that run automatically.

11. Conclusion

The Personal Portfolio Tracker achieves what it set out to do: a working, self-contained investment tracking tool that covers the CSE2006 syllabus from OOP fundamentals through JDBC and file I/O, without relying on any framework beyond the standard library and a single database driver.

More than a checklist of covered topics, the project produced something that can actually be used. Real data can be entered, watched grow or shrink, and exported for analysis. That concreteness made every design decision feel meaningful rather than academic. The challenges - encoding, classpath management, schema design - were the kind that come up in real software work, and working through them was more instructive than any textbook exercise.

The codebase is clean, the architecture is honest about what it is, and the scope is controlled. For a first substantial Java project, that feels like the right outcome.

---

12. References

[1] Herbert Schildt, Java: The Complete Reference, 11th Edition, Oracle Press, 2018.

[2] Deitel & Deitel, Java How to Program, 10th Edition, Pearson Education, 2015.

[3] SQLite JDBC Driver - Taro L. Saito (xerial): https://github.com/xerial/sqlite-jdbc

[4] Oracle Java SE 17 Documentation: https://docs.oracle.com/en/java/javase/17/

[5] SQLite Documentation: https://www.sqlite.org/docs.html

[6] Adoptium OpenJDK Builds: https://adoptium.net